

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
SIMATS ENGINEERING
COMPUTER SCIENCE AND ENGINEERING
CSA17 - ARTIFICIAL INTELLIGENCE

Experiment No. 3: Python Program for the Water Jug Problem

Aim

To write a Python program to solve the Water Jug problem, which involves using two jugs of given capacities to measure out a required target quantity of water, by applying the Breadth-First Search (BFS) technique on the state space of possible jug configurations.

Algorithm

1. Start.
2. Read the capacities of Jug1 (cap1) and Jug2 (cap2), and the target amount of water to be measured.
3. Represent the state of the problem as a pair (x, y), where x is the current amount of water in Jug1 and y is the current amount of water in Jug2.
4. Initialize the start state as (0, 0), meaning both jugs are empty.
5. Define the possible operations on a state: fill Jug1 completely, fill Jug2 completely, empty Jug1, empty Jug2, pour water from Jug1 to Jug2 until Jug2 is full or Jug1 is empty, and pour water from Jug2 to Jug1 until Jug1 is full or Jug2 is empty.
6. Use a queue to implement Breadth-First Search; insert the start state into the queue along with an empty path list, and mark it visited.
7. While the queue is not empty: a) Remove the front state from the queue. b) If either jug contains the target amount, record and return the path taken. c) Otherwise, generate all valid next states using the operations above.
8. For every next state not already visited, mark it visited and insert it into the queue with the updated path.
9. If the queue becomes empty without reaching the target, report that no solution exists.
10. Print each step of the path from the initial state to the state where the target amount is achieved.
11. Stop.

Pseudocode

```
BEGIN
  INPUT: cap1, cap2, target
  start = (0, 0)
  CREATE queue Q, INSERT (start, path=[start])
  CREATE VISITED = { start }

  WHILE Q is not empty DO
    (x, y), path = Q.dequeue()

    IF x == target OR y == target THEN
      PRINT "Solution found"
      PRINT path
      BREAK
    END IF
```

```

next_states = {
    (cap1, y),          // fill Jug1
    (x, cap2),          // fill Jug2
    (0, y),             // empty Jug1
    (x, 0),             // empty Jug2
    pour Jug1 -> Jug2,
    pour Jug2 -> Jug1
}

FOR each state IN next_states DO
    IF state NOT IN VISITED THEN
        ADD state TO VISITED
        Q.enqueue(state, path + [state])
    END IF
END FOR
END WHILE

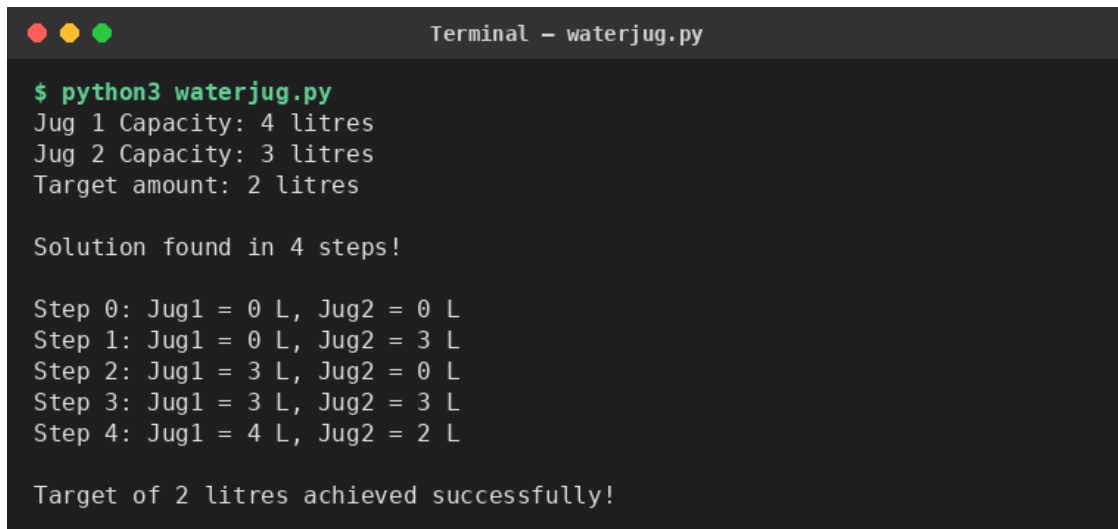
IF target not reached THEN
    PRINT "No solution exists"
END IF
END

```

Result

The Python program for the Water Jug Problem was executed successfully using the Breadth-First Search technique. For jugs of capacity 4 litres and 3 litres with a target of 2 litres, the program explored the state space and found the shortest sequence of fill, empty, and pour operations required to measure the target quantity. The sequence of intermediate jug states was displayed step by step until the target amount was obtained. Thus, the Water Jug problem was solved successfully using Python.

Output Screenshot:



```

Terminal - waterjug.py

$ python3 waterjug.py
Jug 1 Capacity: 4 litres
Jug 2 Capacity: 3 litres
Target amount: 2 litres

Solution found in 4 steps!

Step 0: Jug1 = 0 L, Jug2 = 0 L
Step 1: Jug1 = 0 L, Jug2 = 3 L
Step 2: Jug1 = 3 L, Jug2 = 0 L
Step 3: Jug1 = 3 L, Jug2 = 3 L
Step 4: Jug1 = 4 L, Jug2 = 2 L

Target of 2 litres achieved successfully!

```